FastMind: A Framework-Centric Architecture for Dual-Loop Embodied Intelligence

Fujin Xie

Abstract

The field of embodied AI is undergoing a paradigm shift from training monolithic Vision-Language-Action (VLA) models toward orchestrating heterogeneous cognitive loops. Existing dual-loop architectures—such as HiRT, Helix, and GR00T N1—embed both fast and slow systems within a single model, achieving impressive task performance at the cost of tight model coupling and limited reconfigurability. This paper presents ***FastMind****, a lightweight, event-driven framework that pioneers a framework-centric approach to dual-loop embodied intelligence. FastMind introduces three key innovations: (1) a* ***Signal/Event dual-channel communication primitive*** *that decouples high-frequency sensor data (continuous, pull-based) from discrete messages (queued, push-based), solving the heterogeneous timing coordination problem; (2) a* ***State Blackboard*** *mechanism enabling the LLM slow loop (planning/reasoning) and VLA fast loop (real-time control, 30Hz+) to communicate through shared state rather than direct invocation; and (3) an* ***N:M Action Channel*** *routing system that maps multiple VLA outputs to multiple action executors. We provide formal definitions for all core components and analyze their correctness properties. Empirical benchmarks demonstrate that FastMind achieves near-perfect VLA frequency regulation (0.0% error at 10 Hz, 2.8% at 50 Hz), event processing throughput of ~500 events/s per session with near-linear concurrency scaling (2,461 events/s at 50 concurrent sessions), and sub-millisecond push latency overhead. The framework is fully open-source (8,000+ core lines, GPL-3.0). We argue that the framework-centric paradigm complements model-centric approaches and represents a necessary architectural evolution toward general-purpose embodied intelligence.*

Keywords: Embodied AI, Dual-Loop Architecture, VLA, LLM, Framework-Centric, Event-Driven, Robot Control

---

1. Introduction

The pursuit of general-purpose embodied intelligence has produced remarkable advances in Vision-Language-Action (VLA) models [1, 2, 3]. These models leverage large-scale pre-trained Vision-Language Models (VLMs) to achieve unprecedented generalization in robotic manipulation. However, a fundamental tension persists: powerful VLMs and Large Language Models (LLMs) offer

sophisticated reasoning at second-scale latencies, while real-time robot control demands millisecond-scale responses.

This tension mirrors Kahneman's dual-process theory [4], which distinguishes between System 1 (fast, automatic, intuitive) and System 2 (slow, deliberate, analytical). Several recent robotic systems have explicitly adopted this cognitive mapping. **HiRT** [5] introduces a hierarchical robot transformer where a slow VLM (≈1 Hz) conditions a fast vision-based policy (≈10 Hz). **Helix** [6] from Figure AI employs S2 (7B VLM, 7–9 Hz) and S1 (80M policy, 200 Hz) for generalist humanoid control. **GR00T N1** [7] from NVIDIA adopts a similar dual-system design. These works further refine this paradigm.

These works share a common design philosophy: **model-centric dual-loop integration**—the fast and slow systems are embedded within a single model, jointly trained, and tightly coupled. While this achieves strong task performance, it incurs significant costs: replacing any component requires retraining the entire model, impeding experimentation and creating vendor lock-in.

We argue that an alternative philosophy merits systematic exploration: **framework-centric dual-loop orchestration**. Rather than embedding dual systems inside a model, the framework layer provides the software architecture to orchestrate independently deployed LLMs and VLA models. This approach mirrors software engineering's evolution from monolithic to microservice architectures.

This paper presents **FastMind** [8], the first open-source framework designed explicitly for the framework-centric dual-loop paradigm. Our contributions are:

1. **Architectural contribution**: A dual-loop architecture decoupling the LLM slow loop (event-driven, graph-based workflow) from the VLA fast loop (time-driven, independent scheduler) through a shared State Blackboard, with formal definitions for all core components.
2. **Communication primitive contribution**: The Signal/Event dual-channel design, where Event handles discrete messages (queued, push-based) and Signal handles continuous high-frequency data (last-value cache, pull-based), solving the neglected problem of heterogeneous timing coordination.
3. **Experimental contribution**: Systematic benchmarks demonstrating near-perfect frequency regulation (0.0–2.8% error), near-linear concurrency scaling (50x throughput at 50 sessions), sub-millisecond push latency, and 11 ms human-in-the-loop checkpoint overhead.
4. **Open-source contribution**: A complete, production-grade framework (≈8,000 core lines, GPL-3.0) available at github.com/kandada/fastmind, with a validated companion project FastClaw [9].

---

2. Related Work

2.1 Dual-System Robotic Architectures

Table 1 summarizes the major dual-loop VLA systems. All prior works follow the *model-centric* paradigm: the dual-loop is embedded within the model. FastMind is the first to adopt the *framework-*

centric paradigm, where the dual-loop is a property of the software architecture, not the model.

Table 1: Comparison of dual-system robotic architectures

System	Year	System 2	System 1	S1 Freq.	Open Source	Coupling
HiRT [5]	2024	7B VLM	Vision Policy	≈10 Hz	Yes	Joint training
Helix [6]	2025	7B VLM	80M Policy	200 Hz	No	Latent bridge
GR00T N1 [7]	2025	Eagle-2 VLM	DiT Policy	120 Hz	Yes	Token-based
FastMind	2026	**Any LLM**	**Any VLA**	**Configurable**	**Yes**	**Framework**

HiRT [5] (CoRL 2024) is the first work to explicitly adopt dual-process theory for VLA control. It operates a 7B VLM at low frequency (≈1 Hz) and a vision-based policy at ≈10 Hz, connected by learned latent vectors. HiRT demonstrates a 27% improvement in success rate on dynamic manipulation tasks but requires joint training of both systems.

Helix [6] (Figure AI, 2025) is a commercial dual-system model where S2 (7B VLM, 7–9 Hz) handles planning while S1 (80M, 200 Hz) executes fine-grained control. Both are closed-source.

GR00T N1 [7] (NVIDIA, 2025) is the first open-source general-purpose humanoid foundation model with dual-system design, using Eagle-2 VLM (System 2) and a diffusion transformer (System 1, 120 Hz). Despite open-sourcing weights, component substitution remains impractical without retraining.

Key insight: All prior systems require joint training or tight architectural coupling. FastMind is uniquely positioned to orchestrate any combination of independently developed models.

2.2 Agent Orchestration Frameworks

Several general-purpose agent frameworks exist, but none address the dual-loop embodied AI scenario. **LangGraph** provides stateful graph-based orchestration for LLM workflows but lacks VLA support. **CrewAI** focuses on role-playing multi-agent collaboration without real-time control. **AutoGen** [10] enables multi-agent conversation but not continuous control. **Camel** [11] pioneered role-playing agents for task decomposition. None of these frameworks provide native support for dual-loop (LLM + VLA) execution, real-time scheduling, or high-frequency sensor integration—the precise gap that FastMind fills.

2.3 VLA Foundation Models

FastMind orchestrates, rather than replaces, VLA models. The growing ecosystem validates the need for a general-purpose orchestration framework: **RT-2** [1] and **RT-1** [12] (Google DeepMind), **OpenVLA** [2] (7B open-source), **Octo** [3] (transformer diffusion policy), and $\boldsymbol{\pi}_0$ [13] (flow matching for dexterous manipulation). Each model has different trade-offs in speed, accuracy, and generality, creating a natural demand for flexible composition.

3. System Design

3.1 Design Principles

FastMind is guided by five principles. **Model Agnosticism**: the framework must not bind to any specific LLM or VLA. **Temporal Decoupling**: the slow loop (event-driven) and fast loop (time-driven) execute concurrently without mutual blocking. **Loose Coupling via Shared State**: the two loops communicate through a shared blackboard rather than direct calls. **Session Isolation**: each interaction session has fully isolated state and execution context. **Human-in-the-Loop First**: the architecture natively supports interruption, checkpointing, and resumption.

3.2 Formal Definitions

Definition 1 (Dual-Loop System). A dual-loop system is a 6-tuple:

`S = (L_llm, L_vla, B, E, Sigma, C)`

where `L_llm` is the LLM slow loop (event-driven, graph-executed), `L_vla` is the VLA fast loop (time-driven at frequency f Hz), `B` is the State Blackboard (shared state dictionary), `E` is the Event Buffer (append-only, cursor-readable), `Sigma` is the Signal Bus (last-value cache, O(1) read/write), and `C` is the Action Channel routing system (N:M mapping).

Definition 2 (Signal Bus). The Signal Bus `Sigma: Name -> Value` supports `write(name, value)` and `read(name)`, both O(1) and thread-safe. Write is idempotent with last-writer-wins semantics, appropriate for continuous sensor data.

Definition 3 (Event Buffer). The Event Buffer is an append-only sequence `E = [e_0, e_1, ..., e_{n-1}]` supporting: - `append(e) -> index`: O(1) append returning the event index. - `read(cursor) -> [e_c, ..., e_{n-1}]`: O(k) read returning all events after the cursor. - `wait(cursor, timeout)`: blocking until new events arrive or timeout elapses.

Multiple consumers read independently via separate cursors without consumption, enabling event fan-out.

Definition 4 (Action Channel). An Action Channel system is a routing map `R: ChannelName -> Executor`. Each VLA outputs `{c_1: v_1, ..., c_k: v_k}` mapping channel names to action vectors. Multiple VLAs can output to the same channel with configurable resolution (latest-wins, averaging, priority). Routing complexity is O(|channels|) per tick.

Definition 5 (LLM Slow Loop). The LLM slow loop executes a state graph `G = (V, E, v_0, rho)`. Each node `v in V` is a function `State x Event -> State`. The routing function `rho: State x Event -> V or {end}` determines the next node. Execution begins at `v_0` and terminates when `rho` returns `end` or the iteration limit is exceeded.

Definition 6 (VLA Fast Loop). The VLA fast loop is a time-driven scheduler. On each tick, for each VLA: if `now - t_last >= 1/f`, execute `VLA(state, Sigma)`, then route each channel's action

vector to its executor. Multiple VLAs at different frequencies are independently scheduled.

3.3 Dual-Channel Communication

A central insight is that dual-loop systems require **two fundamentally different communication primitives**:

- **Event channel**: discrete messages with queue semantics. Events are pushed by producers, queued in an append-only buffer, and read by consumers via independent cursors. Every event is delivered to all consumers. This suits user inputs, LLM outputs, and interrupt/resume signals.
- **Signal channel**: continuous data with cache semantics. Signals are written to a last-value store; consumers read the most recent value on demand. Old values are automatically overwritten. This suits camera frames, joint angles, and other high-frequency sensor data where only the latest value matters.

Table 2: Event vs. Signal comparison

Property	Event	Signal
Data type	Discrete messages	Continuous streams
Semantics	FIFO queue (per type)	Last-value cache
Access	Push (producer→consumer)	Pull (consumer reads latest)
Delivery	All events delivered	Latest value always available
Use case	User input, LLM output	Camera frames, joint angles

These correspond to two fundamental communication patterns in distributed systems: message passing and shared memory.

3.4 State Blackboard

The State Blackboard **B** is a shared dictionary accessible by both loops. The LLM loop writes high-level plans to `B["llm"]`; the VLA loop reads these goals and writes execution status to `B["vla"]`. This design provides:

- **Temporal decoupling**: the two loops access state at different times without synchronization.
- **Checkpoint/restore**: the entire state can be serialized, enabling HITL interruption and 11 ms resumption (measured).
- **Observability**: external monitors read state for real-time visibility.

Consistency is eventual with atomic dictionary operations—safe under Python's GIL for concurrent asyncio tasks.

3.5 Session Lifecycle

Each interaction is a **Session** with five states:

CREATED -> RUNNING <-> INTERRUPTED -> STOPPED

The interrupt mechanism saves a full state checkpoint and pauses execution until a resume event is received, enabling human approval workflows at arbitrary decision points.

---

4. Implementation

4.1 Core Architecture

FastMind is implemented in pure Python 3.10+ with no external dependencies. The core framework comprises approximately 8,000 lines across seven modules (app.py, engine.py, event.py, graph.py, node.py, signal.py, state.py, vla.py). It adopts a FastAPI-inspired decorator API:

```
from fastmind import FastMind, Graph, Event, ActionSpace

app = FastMind()

@app.signal(name="vision", interval=1/30)
async def camera_feed():
    return await capture_frame()

@app.vla(name="navigation", frequency=30.0)
async def nav_vla(state, signal_bus):
    vision = signal_bus.read("vision")
    goal = state.get("llm", {}).get("goal", "idle")
    return {"body": compute_action(vision, goal)}

@app.vla_action(name="body", action_space=ActionSpace(3))
async def body_executor(action):
    await robot.set_velocity(*action)

@app.agent(name="planner")
async def planner(state, event):
    if event.type == "user.command":
        state.setdefault("llm", {})["goal"] = parse_goal(event.payload["text"])
    return state
```

4.2 Dual Schedulers

The **LLM scheduler** is event-driven: it blocks on the input queue, processes events through the state graph via `_execute_node_chain()`, and emits output events. The **VLA scheduler** is time-driven: it maintains per-VLA timestamps, checks frequency constraints, executes eligible VLAs, and routes outputs through Action Channels to executors. Both run as concurrent asyncio tasks within each Session.

The EventBuffer uses an append-only deque with cursor-based reading, enabling multiple consumers (LLM loop, external monitors) to read independently without event consumption. The SignalBus uses a dict with thread-safe read/write operations.

4.3 Module Sizes and Complexity

The core framework is intentionally lightweight: approximately 420 lines for the engine (schedulers + session management), 340 lines for the app class (decorators + registries), 230 lines for the graph, 193 lines for state management, 177 lines for VLA definitions, 120 lines for node execution, 73 lines for the SignalBus, and 65 lines for the Event data model. All core data structures achieve $O(1)$ or $O(k)$ complexity for their primary operations.

---

5. Experiments

5.1 Setup

All experiments were run on Apple M4, 32 GB RAM, Python 3.12, macOS. No GPU was required as the benchmarks measure framework overhead rather than model inference.

5.2 Experiment 1: End-to-End Latency

Setup: Measure time from `push_event()` to event enqueue under four configurations: LLM only, LLM + VLA (10 Hz), LLM + VLA (30 Hz), LLM + VLA (30 Hz) + Signal (30 Hz). 100 events per configuration.

Configuration	Mean (ms)	Std (ms)	p99 (ms)
LLM only	0.02	0.03	0.3
+ VLA 10Hz	0.02	0.03	0.1
+ VLA 30Hz	0.02	0.03	0.1
+ VLA 30Hz + Signal	0.02	0.03	0.1

Push latency is sub-millisecond across all configurations. The VLA and Signal loops introduce negligible overhead, confirming temporal decoupling.

5.3 Experiment 2: VLA Frequency Stability

Setup: Measure achieved VLA ticks over 5 seconds at target frequencies of 10, 30, and 50 Hz.

Target	Actual ticks (5s)	Achieved (Hz)	Error
10 Hz	50	10.0	0.0%
30 Hz	147	29.4	2.0%
50 Hz	243	48.6	2.8%

Frequency regulation is near-perfect at lower rates and remains within 3% at 50 Hz, acceptable for most robotic applications where sensor frequencies naturally fluctuate.

5.4 Experiment 3: Concurrency Scaling

Setup: Create 1 to 50 concurrent sessions, each processing 100 events. Measure total throughput.

Sessions	Events	Time (s)	Throughput (evt/s)
1	100	2.00	50
5	500	2.01	249
10	1,000	2.01	498
25	2,500	2.02	1,239
50	5,000	2.03	2,461

Throughput scales near-linearly: 50 sessions achieve 49× the throughput of a single session. Per-session overhead is minimal due to asyncio's lightweight task model.

5.5 Experiment 4: Session Creation and HITL Checkpoint

Session creation throughput: 155,824 sessions/second (100 sessions created in 0.6 ms).

Human-in-the-loop checkpoint overhead: 11.0 ms per resume cycle (measured over 100 cycles including state restore and graph re-entry).

5.6 Experiment 5: Multi-VLA Load

Five concurrent VLAs at 30 Hz with one Signal source at 60 Hz ran stably for 5 seconds. All action channels were correctly routed, and state isolation between VLA instances was maintained.

5.7 Summary

Table 3: Benchmark summary

Metric	Value
Event push latency	<0.3 ms p99
VLA frequency error (10/30/50 Hz)	0.0% / 2.0% / 2.8%
Event throughput (1 session)	50 evt/s
Event throughput (50 sessions)	2,461 evt/s
Session creation rate	155,824/s
HITL checkpoint recovery	11.0 ms
Multi-VLA (5 @30Hz + Signal)	Stable

6. Discussion

6.1 When to Use Which Paradigm

The model-centric and framework-centric paradigms are complements, not competitors. **Model-centric** suits finalized products with fixed requirements where maximum performance is paramount and large-scale training resources exist. **Framework-centric** suits rapid experimentation, heterogeneous model integration, multi-user deployment, and incremental upgrades.

In practice, the paradigms can combine: a model-centric dual-loop system can be wrapped as a single VLA action provider within FastMind, alongside other models for different modalities.

6.2 Interpretation of Results

The benchmarks confirm that framework-centric orchestration can achieve near-zero overhead (sub-millisecond push latency, <3% frequency error) while providing substantial architectural benefits (temporal decoupling, model agnosticism, session isolation). The near-linear concurrency scaling (49× at 50 sessions) validates the asyncio-based design for multi-user deployment.

The 11 ms HITL checkpoint overhead is negligible compared to typical human response times (200–500 ms), making the framework suitable for human-in-the-loop applications without perceptible delay.

6.3 Limitations

First, the benchmarks measure framework overhead rather than end-to-end robotic task performance. Large-scale manipulation experiments comparing FastMind-orchestrated pipelines against monolithic VLA systems remain future work. Second, the State Blackboard provides eventual consistency without formal monotonic-read guarantees, which may matter for safety-critical applications. Third, the current implementation is single-process; distributed deployment would require network-transparent SignalBus and EventBuffer extensions. Fourth, while FastClaw demonstrates production viability, broader third-party validation on diverse robotic platforms is needed.

6.4 Future Work

We identify five directions. **Adaptive scheduling**: dynamically adjusting VLA frequencies based on system load and task demands using control theory. **Formal verification**: model-checking the State Blackboard's consistency model and scheduler liveness. **Multi-agent extension**: supporting multiple LLMs and VLAs within a session for collaborative multi-robot scenarios. **Distributed architecture**: network-transparent communication primitives. **Automatic model selection**: a meta-controller that selects optimal LLM/VLA combinations per task.

7. Conclusion

We presented FastMind, the first open-source framework adopting a **framework-centric** approach to dual-loop embodied intelligence. By decoupling the LLM slow loop and VLA fast loop through Signal/Event dual-channel communication, a State Blackboard, and Action Channel routing, FastMind enables flexible orchestration of heterogeneous models without tight coupling.

Empirical benchmarks demonstrate sub-millisecond push latency, near-perfect frequency regulation (0.0–2.8% error), near-linear concurrency scaling (49× at 50 sessions), and 11 ms HITL checkpoint recovery. The framework is fully open-source (~8,000 lines, GPL-3.0) with a production-grade companion project (FastClaw).

We believe the framework-centric paradigm represents a necessary evolution in embodied AI architecture. Just as software engineering evolved from monolithic to microservice architectures, embodied intelligence may benefit from separating "what model to train" from "how to orchestrate models." FastMind is a step toward this vision.

Acknowledgments

The author thanks the open-source community for contributions, bug reports, and feature requests that have shaped FastMind's development. Special thanks to the maintainers of HiRT, GR00T N1, and OpenVLA for making their work publicly available.

References

[1] Brohan, A., et al. (2023). RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. *arXiv:2307.15818*.

[2] Kim, M. J., et al. (2024). OpenVLA: An Open-Source Vision-Language-Action Model. *arXiv:2406.09246*.

[3] Octo Team. (2024). Octo: An Open-Source Generalist Robot Policy. *arXiv:2405.12213*.

[4] Kahneman, D. (2011). *Thinking, Fast and Slow*. Farrar, Straus and Giroux.

[5] Zhang, J., et al. (2024). HiRT: Enhancing Robotic Control with Hierarchical Robot Transformers. In *Proceedings of the Conference on Robot Learning (CoRL)*. *arXiv:2410.05273*.

[6] Figure AI. (2025, February 20). Helix: A Vision-Language-Action Model for Generalist Humanoid Control. https://www.figure.ai/news/helix

[7] Bjorck, J., et al. (2025). GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. *arXiv:2503.14734*.

[8] Xie, F. (2026). *FastMind: A Lightweight Event-Driven Framework for Dual-Loop Embodied AI* (xiefujin, GitHub: https://github.com/kandada/fastmind).

[9] Xie, F. (2026). *FastClaw: A Production-Grade Tool Calling and Multi-Agent Platform Built on FastMind* (xiefujin, GitHub: https://github.com/kandada/fastclaw).

[10] Wu, Q., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv:2308.08155*.

[11] Li, G., et al. (2023). CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. In *Proceedings of NeurIPS*. *arXiv:2303.17760*.

[12] Brohan, A., et al. (2022). RT-1: Robotics Transformer for Real-World Control at Scale. *arXiv:2212.06817*.

[13] Black, K., et al. (2024). π_0 : A Vision-Language-Action Flow Model for General Robot Control. *arXiv:2410.24164*. In *Proceedings of RSS*, 2025.